

Segundo projeto - Introdução à Física Computacional

Evandro Risso Venâncio¹

¹Instituto de Física de São Carlos - IFSC USP

risso.evandro@usp.br

Introdução

Este relatório tem como objetivo principal apresentar e explicar os códigos desenvolvidos no segundo projeto da matéria de Introdução à Física Computacional, além de demonstrar a aplicação prática do uso do Fortran 77. A maioria dos exercícios foram baseados na ideia de geradores aleatórios utilizando a função *rand()* do Fortran.

Exercício 1

O primeiro exercício do projeto tem como objetivo testar o gerador de números aleatórios do Fortran, utilizando a função *rand()*, que gera pseudo-números aleatórios no intervalo [0,1]. Para isso, foi calculado os **momentos** da distribuição gerada, ou seja:

$$\langle x^n \rangle, \quad \text{para } n = 1, 2, 3, 4$$

Os momentos são utilizados para descrever as propriedades de uma distribuição. Sabendo o intervalo podemos comparar os resultados obtidos com os resultados teóricos esperados. O código desenvolvido está representado a seguir:

```
1  program ex1
2      call media_gerada()
3  end program ex1
4
5  subroutine media_gerada
6      real*8 x, soma1, soma2, soma3, soma4
7      integer N, i
8
9      N = 1000000000
10
11     soma1 = 0.0
12     soma2 = 0.0
13     soma3 = 0.0
14     soma4 = 0.0
15
16     do i = 1, N
17         x = rand()
18
19         soma1 = soma1 + x
20         soma2 = soma2 + x**2
21         soma3 = soma3 + x**3
22         soma4 = soma4 + x**4
23     end do
24
25     print*, 'Media de x^1: ', soma1/N
26     print*, 'Media de x^2: ', soma2/N
27     print*, 'Media de x^3: ', soma3/N
28     print*, 'Media de x^4: ', soma4/N
29 end subroutine media_gerada
```

Neste código, um grande número de amostras ($N = 10^9$) foi utilizado para garantir que as médias ficassem mais próximas dos valores esperados. A variável x é um número aleatório gerado pela função `rand()`. Calculamos os momentos $\langle x \rangle$, $\langle x^2 \rangle$, $\langle x^3 \rangle$ e $\langle x^4 \rangle$ somando os valores ao longo das N iterações e, em seguida, dividindo pelo número total de amostras N .

Os momentos de uma distribuição são definidos pela seguinte expressão:

$$\langle x^n \rangle = \frac{1}{N} \sum_{i=1}^N x_i^n$$

Em que x_i são números aleatórios gerados e N o número total de amostras. Neste caso foram calculados os momentos de $n = 1, 2, 3, 4$. Para uma distribuição no intervalo $[0, 1]$, os valores teóricos dos momentos podem ser calculados a partir da fórmula do valor esperado de uma função de uma variável aleatória contínua:

$$\langle x^n \rangle = \int_0^1 x^n dx$$

Resolvendo essa integral para diferentes valores de n :

- Para $n = 1$:

$$\langle x \rangle = \int_0^1 x dx = \left[\frac{x^2}{2} \right]_0^1 = \frac{1}{2} = 0.5$$

- Para $n = 2$:

$$\langle x^2 \rangle = \int_0^1 x^2 dx = \left[\frac{x^3}{3} \right]_0^1 = \frac{1}{3} \approx 0.3333$$

- Para $n = 3$:

$$\langle x^3 \rangle = \int_0^1 x^3 dx = \left[\frac{x^4}{4} \right]_0^1 = \frac{1}{4} = 0.25$$

- Para $n = 4$:

$$\langle x^4 \rangle = \int_0^1 x^4 dx = \left[\frac{x^5}{5} \right]_0^1 = \frac{1}{5} = 0.2$$

Portanto, os valores teóricos para os momentos de $n = 1, 2, 3, 4$ são:

$$\langle x^1 \rangle = 0.5, \quad \langle x^2 \rangle = \frac{1}{3} \approx 0.3333, \quad \langle x^3 \rangle = \frac{1}{4} = 0.25, \quad \langle x^4 \rangle = \frac{1}{5} = 0.2$$

Ao executar o código com $N = 10^9$ amostras, os seguintes valores foram retornados:

```
1 Media de x^1: 0.50000357688113473
2 Media de x^2: 0.33333610516559264
3 Media de x^3: 0.25000308537203053
4 Media de x^4: 0.20000359627291278
```

Os valores calculados se aproximam bastante dos valores teóricos conforme o número de amostras N aumenta.

Exercício 2

O exercício 2, tem como objetivo resolver o problema dos andarilhos aleatórios (random walker).

A ideia por trás é que cada andarilho, independente de onde esteja, possa dar um passo à direita ou à esquerda. A probabilidade dele ir para a direita é p e para a esquerda seria $q = 1 - p$.

A primeira parte do exercício pede para considerar $p = q = 1/2$, ou seja, a probabilidade do andarilho ir para à esquerda ou à direita é a mesma. E a partir desses resultados, construir um histograma do número de andarilhos $n(x)$ em função de x , com todos os andarilhos partindo da origem ($x = 0$).

O código criado em Fortran para a criação do histograma segue abaixo:

```
1  program ex2
2      integer M, N
3
4      N = 1000
5      write(*, *) 'Digite o numero de andarilhos: '; read(*, *) M
6
7      call random_walk(M, N)
8  end program ex2
9
10 subroutine random_walk(M, N)
11     real*8 p, media, media2
12     integer position
13     dimension histogram(-N:N)
14
15     open(1, file="saida-2-#14668849.out")
16
17     p = 0.5
18
19     do i = 1, M
20         position = 0
21         do j = 1, N
22             if (rand() .GT. p) then
23                 position = position + 1
24             else
25                 position = position - 1
26             end if
27         end do
28         histogram(position) = histogram(position) + 1
29     end do
30
31     do i = -N, N
32         write(1, *) i, histogram(i)
33     end do
34 end subroutine random_walk
```

O código utiliza a ideia de que caso o número aleatório gerado pela função `rand()` do Fortran seja maior que a probabilidade $p = 0.5$, então o andarilho "andou" para a direita, e caso seja menor, ele foi para a esquerda.

O histograma é construído com base na posição final do andarilho, então caso um andarilho terminar naquela posição determinada, o vetor armazenará essa posição contabilizando mais um andarilho naquela posição.

O histograma com a probabilidade de $p = 0.5$ está ilustrado logo abaixo:

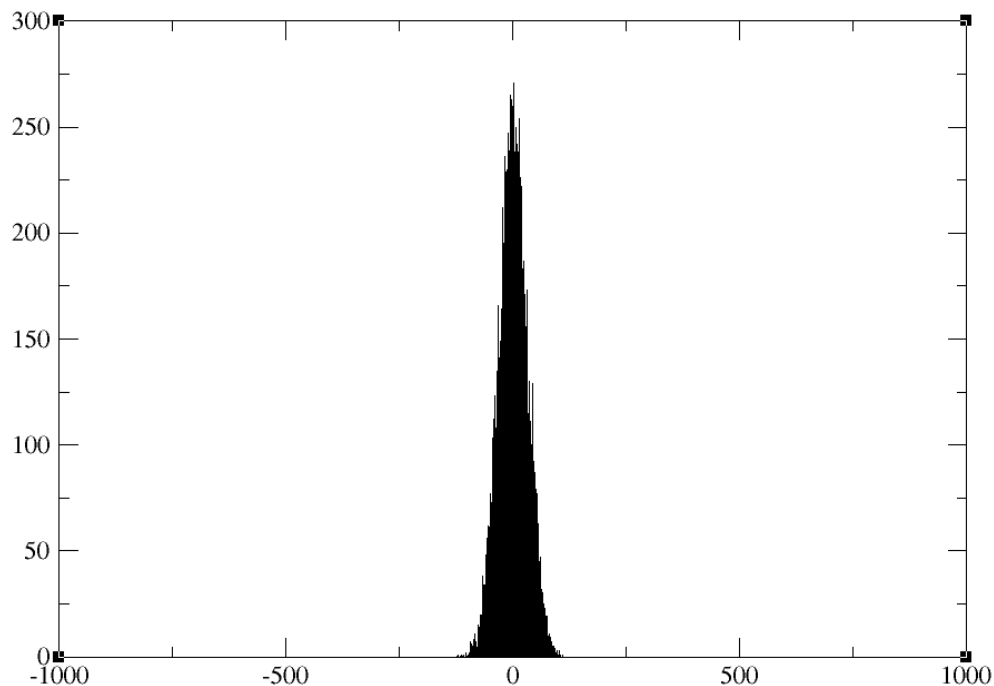


Figura 1. Histograma com probabilidade 0.5 e 10000 andarilhos

O histograma gerado após $N = 1000$ passos para $p = 0.5$ exibe uma curva que se aproxima de uma distribuição normal (gaussiana). Essa curva tem seu pico em torno da posição $x = 0$, refletindo o fato de que a maioria dos andarilhos permanece próxima da origem após os passos. A largura da curva aumenta à medida que aumentamos o número de passos, o que reflete a dispersão dos andarilhos ao longo do tempo.

Após a construção do histograma, calculamos os momentos da distribuição para entender melhor o comportamento dos andarilhos. Primeiramente foi calculado a média das posições finais $\langle x \rangle$, que representa a posição média de todos os andarilhos após N passos.

Além disso, foi calculado a média quadrática $\langle x^2 \rangle$, que nos dá uma medida da dispersão, ou seja, o quanto as posições dos andarilhos variam em torno da média.

No caso do passeio aleatório com $p = 0.5$, espera-se que $\langle x \rangle$ seja próximo de zero, e que $\langle x^2 \rangle$ cresça proporcionalmente ao número de passos N .

O código abaixo realiza esses cálculos:

```
1  subroutine random_walk(M, N)
2      real*8 p, media, media2
3      integer position
4      dimension histogram(-N:N)
5
6      media = 0.0
7      media2 = 0.0
8      do i = -N, N
9          if (histogram(i) .gt. 0) then
10             media = media + i * histogram(i)
11             media2 = media2 + i * i * histogram(i)
```

```

12     end if
13     write(1, *) i, histogram(i)
14 end do
15
16 write(*, *) 'Media: ', media / M
17 write(*, *) 'Media^2: ', media2 / M
18
19 end subroutine random_walk

```

O resultado para 10000 andarilhos e 1000 passos foram:

```

1 Media:  -9.1600000000000001E-002
2 Media^2:  999.86559999999997

```

Esses resultados estão de acordo com o esperado, já que a média $\langle x \rangle$ é próxima de zero e valor de $\langle x^2 \rangle$, é proporcional ao número de passos.

Para a segunda parte do exercício, consideramos os casos com $p \neq 0.5$, onde a probabilidade de caminhar para a direita não é a mesma que para a esquerda. Nessas simulações, esperamos que a média $\langle x \rangle$ não seja mais zero, já que há uma tendência de os andarilhos se moverem mais em uma direção.

Os resultados obtidos para $p = 1/3$ e o histograma foram:

```

1 Media:  333.355200000000002
2 Media^2:  112015.212000000000

```

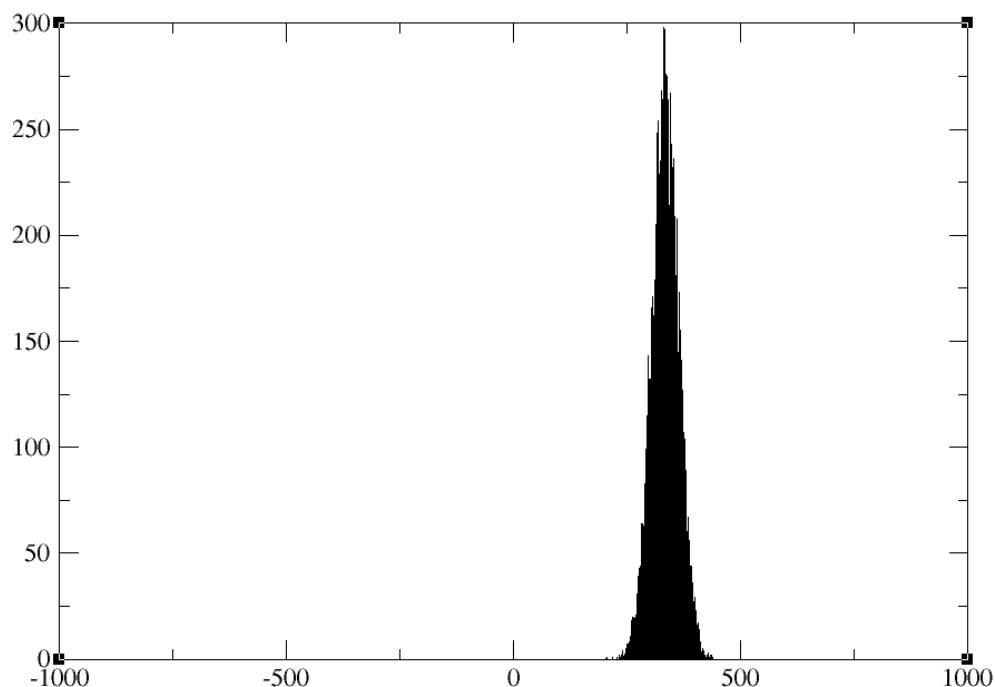


Figura 2. Histograma com 10000 andarilhos e 1000 passos ($p = 1/3$)

Para $p = 1/4$:

```

1 Media:  499.930400000000002
2 Media^2:  250685.761600000000

```

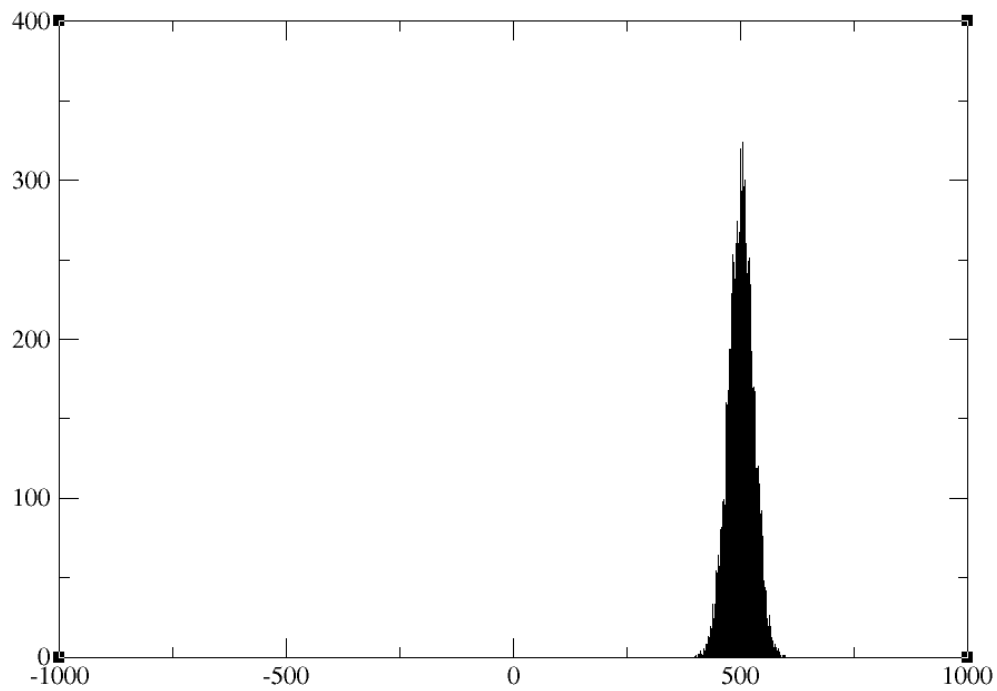


Figura 3. Histograma com 10000 andarilhos e 1000 passos ($p = 1/4$)

Para $p = 1/5$:

```
1 Media: 599.99279999999999
2 Media^2: 360637.36239999998
```

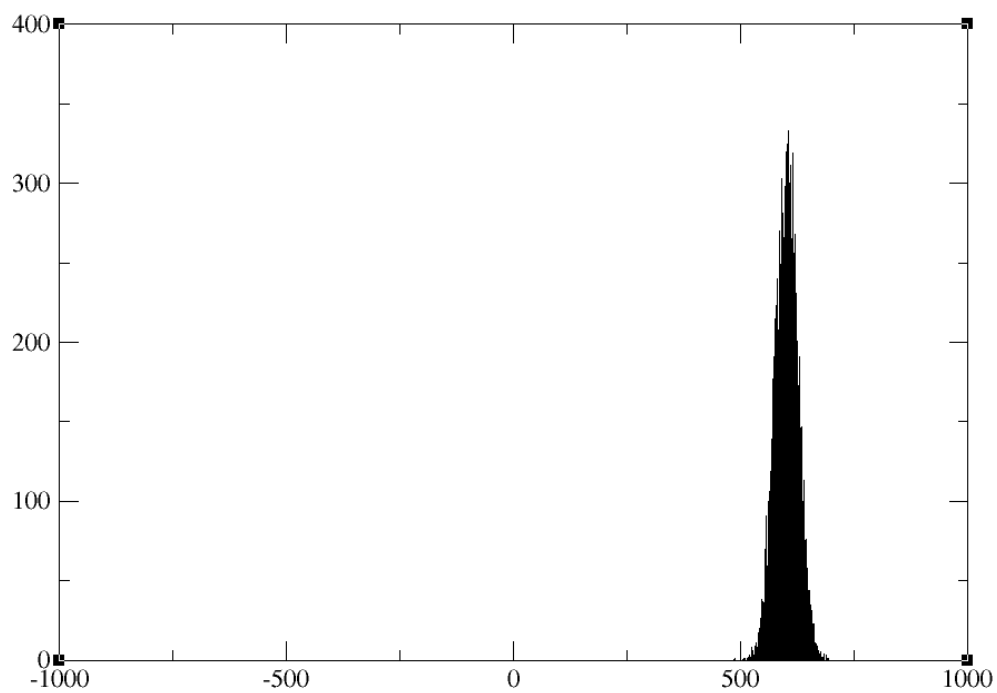


Figura 4. Histograma com 10000 andarilhos e 1000 passos ($p = 1/5$)

Em conclusão, o estudo dos andarilhos aleatórios em uma dimensão mostra como a probabilidade de caminhar em diferentes direções afeta a dispersão e a posição média dos andarilhos. No caso simétrico ($p = 0.5$), os andarilhos tendem a permanecer próximos da origem, com uma dispersão que aumenta com o número de passos. Já no caso assimétrico, observamos uma tendência clara dos andarilhos em se moverem na direção com maior probabilidade, resultando em uma média $\langle x \rangle$ não nula. O código completo do exercício está ilustrado abaixo:

```
1  program ex2
2      integer M, N
3
4      N = 1000
5      write(*, *) 'Digite o numero de andarilhos: '; read(*, *) M
6
7      call random_walk(M, N)
8  end program ex2
9
10 subroutine random_walk(M, N)
11     real*8 p, media, media2
12     integer position
13     dimension histogram(-N:N)
14
15     open(1, file="saida-2-#14668849.out")
16
17     p = 0.5
18     histogram = 0
19
20     do i = 1, M
21         position = 0
22         do j = 1, N
23             if (rand() .GT. p) then
24                 position = position + 1
25             else
26                 position = position - 1
27             end if
28         end do
29         histogram(position) = histogram(position) + 1
30     end do
31
32     media = 0.0
33     media2 = 0.0
34     do i = -N, N
35         if (histogram(i) .gt. 0) then
36             media = media + i * histogram(i)
37             media2 = media2 + i * i * histogram(i)
38         end if
39         write(1, *) i, histogram(i)
40     end do
41
42     write(*, *) 'Media: ', media / M
43     write(*, *) 'Media^2: ', media2 / M
44
45 end subroutine random_walk
```

Exercício 3

O exercício 3 tem como objetivo estudar o comportamento de andarilhos aleatórios bidimensionais, onde cada andarilho pode se mover com chances iguais para norte, sul, leste ou oeste ($\frac{1}{4}$ para cada direção). O objetivo é calcular a posição média $\langle \vec{r} \rangle$ e a dispersão $\Delta^2 = \langle \vec{r} \cdot \vec{r} \rangle - \langle \vec{r} \rangle \cdot \langle \vec{r} \rangle$, que corresponde à variância do movimento. A simulação foi realizada para diferentes números de passos N , como: $N = 10, 10^2, 10^3, 10^4, 10^5, 10^6$. Abaixo está o código que realiza a simulação dos andarilhos bidimensionais:

```
1  program ex3
2      N = 10
3      write(*, *) 'Digite o numero de andarilhos: '; read(*, *) M
4
5      call random_walk_bi(M, N)
6  end program ex3
7
8  subroutine random_walk_bi(M, N)
9      real*8 p, p2, p3, pos, mediaX, mediaY, deltaR, mediaR2
10     dimension ipositionX(M), ipositionY(M)
11
12     p = 0.25
13     p2 = 2 * p
14     p3 = 3 * p
15     deltaR = 0.0
16     mediaX = 0.0
17     mediaY = 0.0
18     mediaR2 = 0.0
19
20     open(1, file="saida-3-#14668849.out")
21
22     do i = 1, M
23         ipositionX(i) = 0
24         ipositionY(i) = 0
25         do j = 1, N
26             pos = rand()
27             if (pos .lt. p) then
28                 ipositionX(i) = ipositionX(i) + 1
29             else if (pos .lt. p2) then
30                 ipositionX(i) = ipositionX(i) - 1
31             else if (pos .lt. p3) then
32                 ipositionY(i) = ipositionY(i) + 1
33             else
34                 ipositionY(i) = ipositionY(i) - 1
35             end if
36
37             write(1, *) ipositionX(i), ipositionY(i)
38         end do
39
40         mediaX = mediaX + ipositionX(i)
41         mediaY = mediaY + ipositionY(i)
42
43         mediaR2 = mediaR2 + (ipositionX(i)**2 + ipositionY(i)**2)
44     end do
45
46     mediaX = mediaX / M
47     mediaY = mediaY / M
```

```

48     mediaR2 = mediaR2 / M
49     deltaR = mediaR2 - (mediaX**2 + mediaY**2)
50
51     write(*, *) 'Media X: ', mediaX
52     write(*, *) 'Media Y: ', mediaY
53     write(*, *) 'Delta^2: ', deltaR
54
55 end subroutine random_walk_bi

```

Dentro da subrotina, as seguintes variáveis são inicializadas:

- p , $p2$, $p3$: São as probabilidades que determinam as quatro direções possíveis de movimento. $p = 0.25$ representa a probabilidade de mover-se para a direita (leste), $p2 = 0.5$ representa a probabilidade acumulada de mover-se para a esquerda (oeste), e $p3 = 0.75$ para mover-se para o norte. Qualquer valor maior que $p3$ corresponde a um movimento para o sul.
- `ipositionX` e `ipositionY`: Arrays que guardam as posições x e y dos andarilhos. Inicialmente, todos os andarilhos começam na origem (0, 0).
- `mediaX`, `mediaY`: Acumulam as posições médias dos andarilhos nas direções x e y .
- `mediaR2`: Acumula o valor de $\langle r^2 \rangle$, ou seja, o quadrado da distância radial total de cada andarilho.
- `deltaR`: Calcula a variância $\Delta^2 = \langle r^2 \rangle - \langle \vec{r} \rangle \cdot \langle \vec{r} \rangle$.

Se `pos` for menor que 0.25, o andarilho move-se para a direita ($x = x + 1$). Se for menor que 0.5, move-se para a esquerda ($x = x - 1$). Valores entre 0.5 e 0.75 resultam em um movimento para o norte ($y = y + 1$), e valores maiores que 0.75 fazem o andarilho mover-se para o sul ($y = y - 1$).

A cada passo, as novas posições dos andarilhos são gravadas em um arquivo. O resultado de 10000 andarilhos e 10 passos, juntamente com seu diagrama está abaixo:

```

1  Media X:      1.7800000000000000E-002
2  Media Y:      2.0000000000000000E-002
3  Delta^2:      9.8324831599999989

```

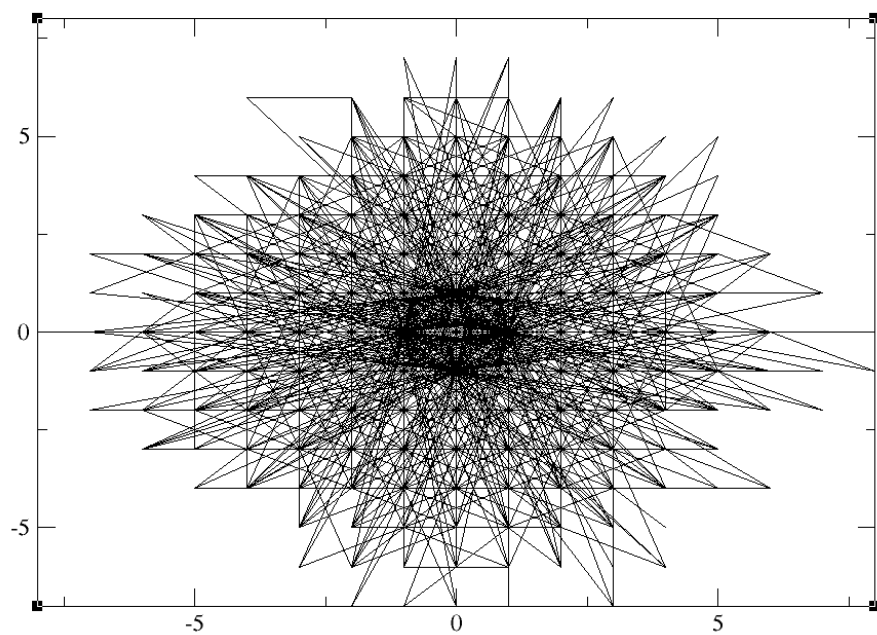


Figura 5. Diagrama de 10000 andarilhos e 10 passos

Para 100 passos:

1	Media X:	2.2100000000000002E-002
2	Media Y:	7.9299999999999995E-002
3	Delta^2:	100.97102310000000

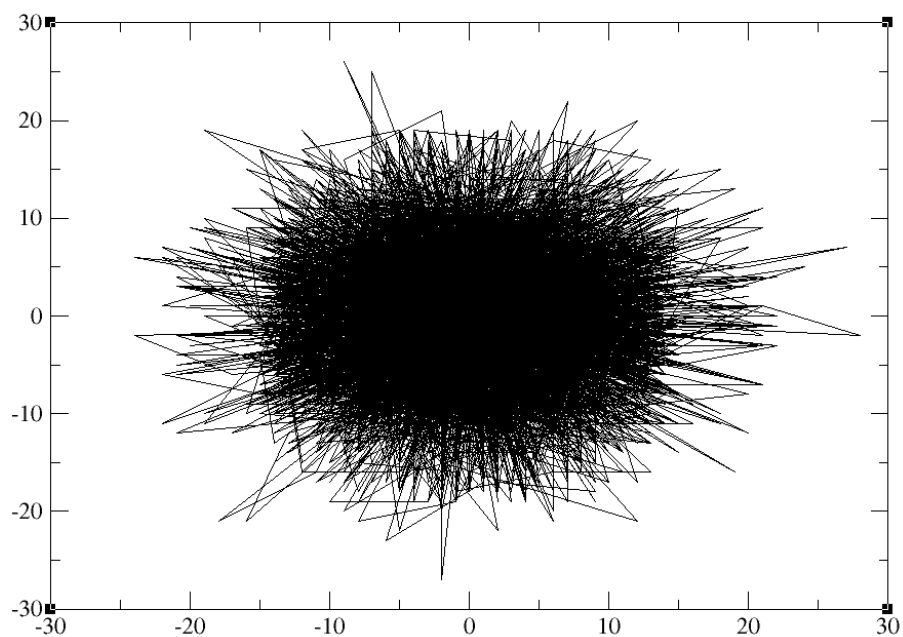


Figura 6. Diagrama de 10000 andarilhos e 100 passos

Para 1000 passos:

```
1 Media X: 2.3199999999999998E-002
2 Media Y: -0.10600000000000000
3 Delta^2: 1015.2246257600000
```

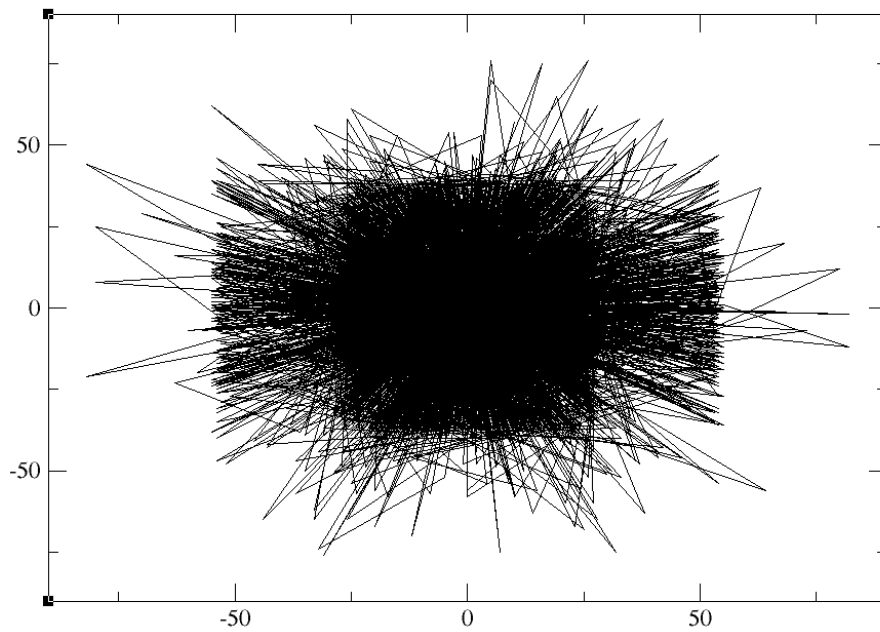


Figura 7. Diagrama de 10000 andarilhos e 1000 passos

Para 10000 passos:

```
1 Media X: 0.23200000000000001
2 Media Y: -1.0600000000000001
3 Delta^2: 10164.738576000000
```

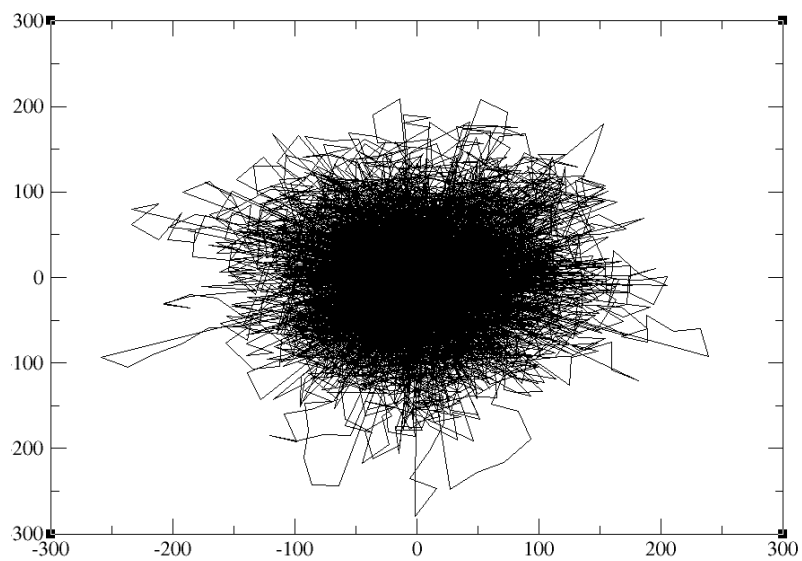


Figura 8. Diagrama de 1000 andarilhos e 10000 passos

Para 100000 passos:

```
1 Media X: -3.6170000000000000
2 Media Y: -4.1810000000000000
3 Delta^2: 98366.558550000002
```

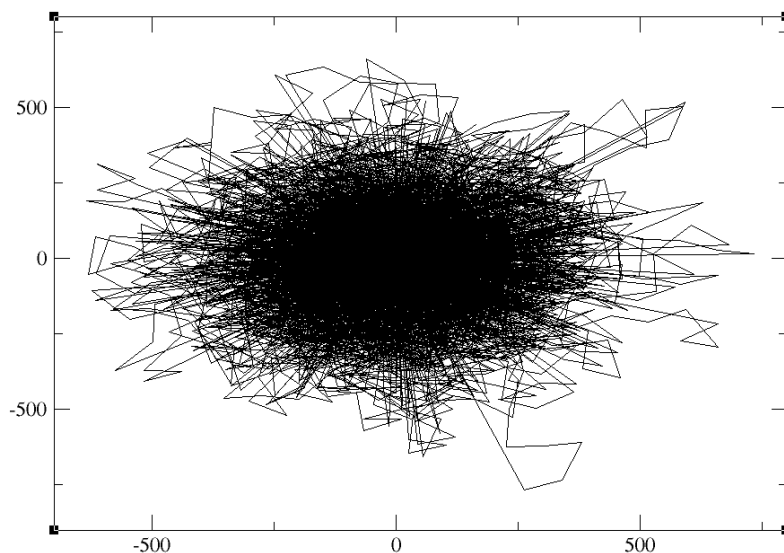


Figura 9. Diagrama de 1000 andarilhos e 100000 passos

Para 1000000 passos:

```
1 Media X: -36.170000000000002
2 Media Y: -41.810000000000002
3 Delta^2: 1147897.7550000001
```

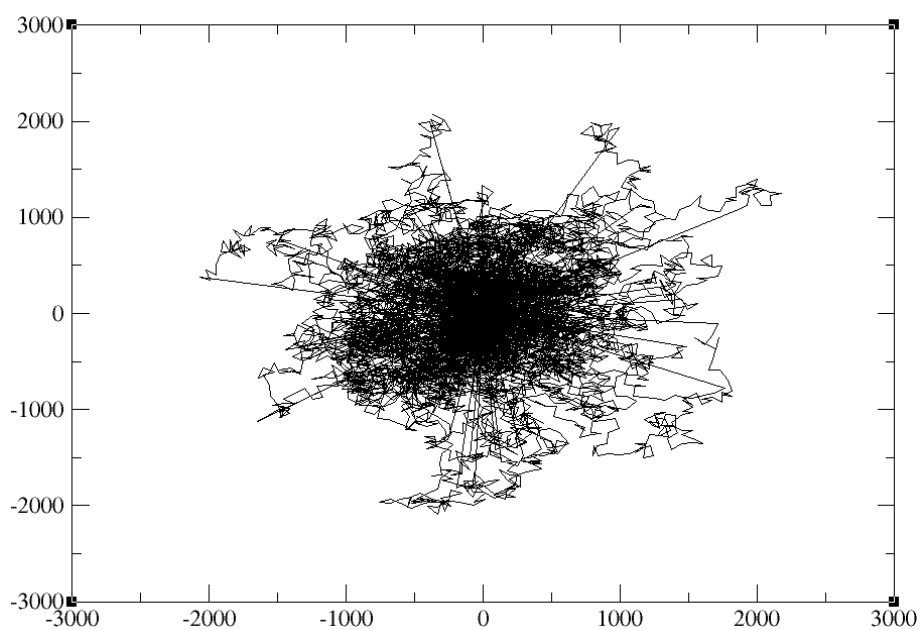


Figura 10. Diagrama de 100 andarilhos e 1000000 passos

É possível perceber que as médias das posições x e y são sempre próximas de zero, o que é esperado de uma simulação simétrica com probabilidades iguais ($p = 1/4$). A variância Δ^2 é da ordem de N , indicando que a dispersão dos andarilhos aumenta proporcionalmente ao número de passos.

Os diagramas resultantes ilustram a concentração de andarilhos próxima ao ponto de origem. As regiões centrais apresentam a maior densidade de andarilhos, refletindo a tendência dos andarilhos de permanecerem relativamente próximos ao ponto de partida após um número de passos.

Exercício 4

O último exercício do projeto tem como objetivo analisar o aumento de entropia e da flecha do tempo em um sistema de andarilhos aleatórios bidimensionais. A entropia S é calculada como uma função do número de passos N , sendo proporcional ao tempo $t = N\Delta t$, onde Δt é o intervalo de tempo médio entre os passos.

A fórmula para calcular a entropia é dada por:

$$S = - \sum_i P_i \ln P_i,$$

onde P_i é a probabilidade de se encontrar o sistema em um certo microestado i . Para definir o microestado, é utilizado um reticulado bidimensional no qual verificamos a quantidade de andarilhos. Essa quantidade de andarilhos em cada reticulado serve para calcular as probabilidades P_i , que são então usadas para determinar a entropia do sistema. O código a seguir realiza a simulação do andarilho aleatório bidimensional e calcula a entropia do sistema após N passos:

```

1  program ex4
2      integer M, N
3
4      N = 1000
5
6      write(*, *) 'Digite o numero de andarilhos: '; read(*, *) M
7
8      call random_walk_bi(M, N)
9
10     end program ex4
11
12     subroutine random_walk_bi(M, N)
13         real*8 p, p2, p3
14         integer positionX, positionY
15         integer grid(-N:N, -N:N)
16
17         p = 0.25
18         p2 = 2*p
19         p3 = 3*p
20
21         do i = 1, M
22             positionX = 0
23             positionY = 0
24             do j = 1, N
25                 pos = rand()

```

```

26
27         if (pos .lt. p) then
28             positionX = positionX + 1
29         else if (pos .lt. p2) then
30             positionX = positionX - 1
31         else if (pos .lt. p3) then
32             positionY = positionY + 1
33         else
34             positionY = positionY - 1
35         end if
36         grid(positionX, positionY) = grid(positionX, positionY) + 1
37     end do
38 end do
39
40 call calculate_entropy(M, N, grid)
41
42 end subroutine random_walk_bi
43
44 subroutine calculate_entropy(M, N, grid)
45     integer M, N, grid(-N:N, -N:N)
46     real*8 entropy, p_i
47
48     entropy = 0.0
49
50     do i = -N, N
51         do j = -N, N
52             if (grid(i, j) .gt. 0) then
53                 p_i = dble(grid(i, j)) / dble(M * N)
54                 entropy = entropy - p_i * log(p_i)
55             end if
56         end do
57     end do
58
59     write(*, *) 'Entropia: ', entropy
60 end subroutine calculate_entropy

```

Cada andarilho começa sua trajetória na origem (0, 0) e realiza N passos. Em cada passo, um número aleatório é gerado utilizando a função `rand()`, e o andarilho se move em uma das quatro direções possíveis:

- Se $\text{pos} < p$, o andarilho se move para a direita (leste).
- Se $p < \text{pos} < 2p$, o andarilho se move para a esquerda (oeste).
- Se $2p < \text{pos} < 3p$, o andarilho se move para cima (norte).
- Se $\text{pos} > 3p$, o andarilho se move para baixo (sul).

A cada passo, o contador da célula correspondente no grid `grid(positionX, positionY)` é aumentado.

A entropia é calculada utilizando as probabilidades P_i associadas à ocupação de cada reticulado no grid. A probabilidade P_i de encontrar o sistema em um determinado micro-estado i é dada pela razão entre o número de andarilhos naquele reticulado e o número total de andarilhos.

Para cada célula (i, j) do grid que contém ao menos um andarilho ($\text{grid}(i, j) > 0$), a probabilidade P_i é calculada e usada para a entropia por meio da fórmula já apresentada. O teste foi realizado para $N = 1000$ passos e $M = 10000$ andarilhos. O valor da entropia

obtido foi:

```
1 Entropia:      8.3069441582735308
```

Esse valor de entropia representa a distribuição de probabilidade associada ao sistema após os N passos dos andarilhos. Um valor mais alto de entropia indica uma maior dispersão dos andarilhos pelo grid, o que corresponde a um sistema mais "desordenado". Com o aumento de N , espera-se que a entropia aumente, indicando a tendência natural do sistema de evoluir para um estado de maior desordem, o que está relacionado à flecha do tempo.

Outro teste agora com $N = 10000$ passos e $M = 10000$ foi realizada:

```
1 Entropia:      10.588886646405319
```

Ou seja, a desordem do sistema aumentou, demonstrando assim que ao aumentar o número de passos a entropia pode aumentar.